



DOSSIÊ TÉCNICO COMPLETO

Sistema Logística Casa do Campo

Arquitetura, fluxos operacionais, banco de dados, segurança,
integração ERP, operação Windows, PWA e aplicativo Android

Snapshot do código: 25/08/2026 | Versão: v2.6.0-20260820 |
Commit: 3366ac8

Como usar este documento

Objetivo principal

Este dossie descreve o sistema que existe no repositório, inclusive divergências entre documentação antiga, código, banco real e projeto Android. Ele não substitui o envio dos arquivos-fonte envolvidos em um bug; serve para dar ao ChatGPT o mapa correto antes da análise pontual.

Escopo analisado: 586 arquivos no repositório, com 112 arquivos de código/documentação operacional relevantes e aproximadamente 28 mil linhas fora de dependências, backups, logs e artefatos. Foram inspecionados o backend Python, a camada modular, o esquema SQLite ativo, as migrations, a integração ERP, a interface web, a PWA do motorista, o wrapper Android, os scripts operacionais e os testes.

O que este PDF contém

- Fluxo ponta a ponta de pedidos, faturamento, cargas, saída, entrega, problema e acerto.
- Arquitetura de execução real: Flask funciona como proxy para o servidor HTTP legado onde as regras ainda vivem.
- Mapa de pastas e responsabilidades de programação.
- Modelo de dados do banco ativo e pontos de drift em relação ao ORM/Alembic.
- Contratos da API do motorista e funcionamento detalhado da PWA e do APK.
- Diagnósticos confirmados e riscos priorizados para correção de bugs.
- Comandos de instalação, teste, backup, restauração e coleta de evidências.
- Um prompt pronto e um checklist de arquivos para conversar com o ChatGPT.

Limites e proteção de dados

Nenhuma senha, segredo, CPF/CNPJ, telefone, endereço, nome de cliente, credencial de ERP ou conteúdo de comprovante foi reproduzido. As contagens do banco aparecem apenas como inventário anônimo. Antes de enviar logs ou trechos do banco a uma IA, remova dados pessoais e segredos.

Sumario

Como usar este documento	2
O que este PDF contém	2
Limites e protecao de dados	2
Sumario	3
1. Visao geral do sistema	6
1.1 Objetivo de negocio	6
1.2 Usuarios e canais	6
1.3 Fluxo operacional ponta a ponta	6
1.4 Estado do snapshot analisado	7
2. Arquitetura e runtime real	8
2.1 Diagrama de componentes	8
2.2 Inicializacao	8
2.3 Implicacoes para manutencao	8
2.4 Stack	8
3. Mapa do repositorio e programacao	9
3.1 Estrutura principal	9
3.2 Fonte de verdade por tipo de bug	9
3.3 Arquivos duplicados ou gerados	9
4. Fluxos e regras de negocio	10
4.1 Pedido	10
Criacao	10
Faturamento e carga	10
Saida e acerto	10
4.2 Carga/rota	10
4.3 SLA	11
4.4 Modulos/telas	11
5. Banco de dados	12
5.1 Entidades operacionais	12
5.2 Seguranca, configuracao e integracao	12
5.3 Relacionamentos essenciais	12
5.4 Drift confirmado	13
6. Seguranca, autenticacao e permissoes	14
6.1 Web administrativo	14
6.2 Perfis e 32 permissoes	14

6.3 Excecao: API motorista	14
7. Integracao ERP	15
7.1 Drivers e prioridade	15
7.2 Ciclos	15
7.3 Pontos de diagnostico	15
8. Aplicativo do motorista: arquitetura completa	16
8.1 Componentes	16
8.2 Inicializacao nativa	16
8.3 Fluxo PWA	16
8.4 Captura de comprovante	16
8.5 Offline	17
8.6 Build e instalacao	17
9. Contrato da API do motorista	18
9.1 Payload de entrega	18
9.2 Efeitos de uma entrega normal	18
10. Diagnosticos confirmados e riscos priorizados	19
10.1 Exemplo do erro P0 reproduzido	19
10.2 Ordem recomendada de correcao	19
11. Testes e qualidade	20
11.1 Resultado verificado	20
11.2 Comando isolado	20
11.3 Cobertura relevante	20
11.4 Testes que devem acompanhar os bugs do app	20
12. Configuracao, instalacao e operacao	21
12.1 Variaveis essenciais	21
12.2 Instalacao Windows	21
12.3 Backup e restauracao	21
12.4 Logs e monitoramento	22
13. Guia de diagnostico do app Android	23
13.1 Arvore de decisao	23
13.2 Evidencias para coletar	23
13.3 Comandos uteis	23
14. Pacote pronto para usar no ChatGPT	24
14.1 Prompt-base	24
14.2 Arquivos por categoria	24
14.3 Formato de relato	25

15. Plano de manutencao recomendado	26
15.1 Curto prazo	26
15.2 Medio prazo	26
15.3 Criterios de pronto para cada bug	26
A. Apendice - Inventario de rotas	27
B. Apendice - Comandos de operacao	28
Conclusao	29

1. Visao geral do sistema

O Logistica Casa do Campo e um sistema interno para coordenar o ciclo de pedidos e distribuicao, com cadastro mestre, faturamento, roteirizacao, operacao de motorista, SLA, acerto, relatorios, permissoes, auditoria e backup.

1.1 Objetivo de negocio

- Transformar pedidos de venda em entregas rastreaveis e encerradas.
- Impedir cargas sem motorista, veiculo, rota ou capacidade valida.
- Controlar o peso total contra a capacidade em kg.
- Calcular prazo de SLA em dias uteis e destacar risco/atraso.
- Registrar historico, usuario, origem e alteracoes de status.
- Permitir operacao de campo por navegador/PWA ou APK WebView.
- Integrar dados do ERP em modo somente leitura e manter cache local.

1.2 Usuarios e canais

Canal	Usuario	Funcao
Web administrativo	GOD, Admin, Gestor, Faturamento, Expedicao, Operador, Consulta	Opera pedidos, cargas, catalogos, SLA, relatorios, configuracoes e backup conforme RBAC.
PWA motorista	Motorista em campo	Seleciona identidade, ve cargas, inicia rota, registra entrega/problema, foto, assinatura e fila offline.
APK Android	Motorista em campo	WebView que carrega a mesma PWA; acrescenta seletor de servidor, QR nativo e integracao de camera/galeria.
ERP	Processo de background e GOD	Fonte read-only de pedidos, clientes, vendedores, produtos e faturamento; dados sao cacheados localmente.
Windows Server	Administrador tecnico	Executa Waitress/monitor, tarefas agendadas, backup, healthcheck e logs.

1.3 Fluxo operacional ponta a ponta



Fluxo normal do pedido: Venda -> Faturado -> Saiu para entrega -> Acertado. Agendado funciona como desvio temporario que pode voltar para Venda, Faturado ou Saiu para entrega conforme NF e carga. Problema e Cancelado sao finais, exceto por reabertura controlada. A carga segue Planejada -> Em rota -> Acertada ou Com problema; Cancelada tambem e final.

1.4 Estado do snapshot analisado

Indicador	Valor anonimo
Banco principal	data/logistica_casa_do_campo.sqlite3 (SQLite WAL)
Integridade	PRAGMA integrity_check = ok
Pedidos	131
Cargas	29
Vinculos carga-pedido	112
Clientes	120
Motoristas	8
Veiculos	6
Historicos de pedido	574
Auditorias	891
Cache ERP de pedidos	17.536
Cache ERP de faturamento	14.288

2. Arquitetura e runtime real

A arquitetura declarada é Flask, mas o comportamento atual é híbrido: Flask recebe todas as URLs e as encaminha a um servidor HTTP legado interno. Portanto, corrigir apenas uma rota Flask normalmente não altera a regra de negócio que ainda está em app.py.

2.1 Diagrama de componentes

Cliente	Entrada pública	Bridge	Aplicação real	Dados/externos
Browser web PWA motorista APK WebView	Waitress :3000 Flask catch-all	LegacyProxyRuntime HTTP 127.0.0.1:4000	App(BaseHTTPRequestHandle r) dispatchers + metodos	SQLite WAL ERP read-only logs/backups

2.2 Inicialização

- run.py importa app.py e cria o objeto Flask com create_app().
- APP_RUNTIME=flask inicia Waitress; APP_RUNTIME=legacy inicia diretamente o SafeThreadingHTTPServer.
- Na primeira requisição, LegacyProxyRuntime inicia um servidor interno na porta APP_PORT + 1000 (padrão 4000).
- O proxy repassa método, corpo, headers e X-Forwarded-* para o servidor interno.
- create_server() chama init_db(), garante esquema/índices/defaults e inicia a thread de sincronização ERP.
- Cada conexão SQLite ativa foreign_keys, busy_timeout=15000, WAL, synchronous=NORMAL e temp_store=MEMORY.

2.3 Implicações para manutenção

Ponto de entrada não é o dono das regras

run.py e app_core/app_factory.py controlam hospedagem e proxy. Autenticação web, HTML, validações, transições, SQL e a maior parte dos endpoints estão em app.py. Os dispatchers em app_core/domains reduzem o roteamento, mas chamam métodos do handler legado.

2.4 Stack

Camada	Tecnologia / versão declarada	Observação
Backend	Python 3.10+, Flask 3+, Waitress 3+	Runtime padrão híbrido Flask + proxy legado.
HTTP legado	ThreadingHTTPServer	Limite por semáforo, fila 128, timeout configurável.
Dados	sqlite3; SQLAlchemy 2; Alembic 1.13+	Regra ativa ainda usa SQL sqlite3; PostgreSQL não é transparente no runtime legado.
Frontend web	HTML gerado em Python, CSS e JavaScript vanilla	templates/ existe, mas a maior UI operacional é montada em app.py.
PWA	HTML/CSS/JS, Service Worker, Web App Manifest	QR web depende de html5-qrcode carregado externamente.
Android	Kotlin 1.8.20, AGP 8.1.0, compile/target 33, min 21	WebView + ZXing + FileProvider.
ERP	Oracle, SQL Server, MySQL, PostgreSQL	Conector read-only, cache local e sync de faturamento.
Operação	PowerShell, Batch, Task Scheduler/NSSM, Caddy opcional	Monitor, backup, firewall, proxy e runbooks.

3. Mapa do repositório e programação

A manutenção correta depende de saber qual arquivo e fonte de verdade, qual o gerador e qual a apenas compatibilidade. Esta seção é o índice técnico para localizar uma mudança.

3.1 Estrutura principal

raiz/	
run.py	entrada Waitress/Flask ou legado
app.py	7.315 linhas: UI, HTTP, regras e SQL ativos
app_core/	
app_factory.py	Flask catch-all e proxy interno
config.py	.env, paths, segurança e limites
runtime_db.py	resolução SQLite/PostgreSQL e backup engine
route_registry.py	telas GET estáveis e permissões
domains/	dispatchers por domínio e API motorista
services/	auditoria, backup, cache, permissões, alertas
repositories/	acesso incremental a users/orders/routes
erp_connector.py	conexão, cache, sync e lookup ERP
erp_field_mapper.py	tradução do ERP para o domínio local
static/	
app.js, style.css	frontend administrativo
driver_app/	PWA do motorista
android_app_project/	projeto Android compilável no Android Studio
migrations/	revisions Alembic 0001..0003
tools/	instalação, testes, backup, monitor e auditorias
tests/	51 testes unittest
data/, backups/, logs/	estado operacional; não enviar sem sanitizar

3.2 Fonte de verdade por tipo de bug

Sintoma	Arquivos a abrir primeiro	Depois verificar
Tela web / botão / permissão	app.py, static/app.js, static/style.css	app_core/domains/*_dispatch.py, permission_service.py
Regra de pedido/carga	app.py	repositories, esquema SQLite, testes correspondentes
ERP	app_core/erp_connector.py, erp_field_mapper.py	settings do banco, .env, logs e testes ERP
App motorista	static/driver_app/driver.js, index.html	driver_api_dispatch.py e banco real
Camera/galeria/QR nativo	android_app_project/app/src/main/.../MainActivity.kt	AndroidManifest.xml, file_paths.xml, Logcat
Build APK	android_app_project/app/build.gradle	build.gradle raiz, settings.gradle, wrapper e tools/build_android_apk.py
Inicialização/porta	run.py, app_factory.py, config.py	iniciar.bat, server_monitor.py e logs
Backup/restore	backup_service.py, runtime_db.py	tools/backup_automation.py e runbooks

3.3 Arquivos duplicados ou gerados

Android: edite o caminho sob app/

Existem duas cópias de MainActivity.kt e AndroidManifest.xml. O módulo Gradle inclui 'app', portanto a compilação usa android_app_project/app/src/main/.... A cópia android_app_project/src/main/... é antiga e divergente. tools/build_android_apk.py regenera o projeto e pode sobrescrever mudanças manuais; ele deve ser atualizado junto com os arquivos gerados.

4. Fluxos e regras de negocio

As transicoes sao validadas no backend. Botoes e telas apenas orientam; a regra efetiva esta em `ensure_order_status()`, `ensure_route_status()` e nos metodos de operacao em `app.py`.

4.1 Pedido

Status	Pode seguir para	Observacoes
Venda	Faturado, Problema, Cancelado, Agendado	Novo pedido sem NF e forçado para Venda.
Faturado	Saiu para entrega, Acertado, Problema, Cancelado, Agendado	Elegivel para carga; NF deve ser unica.
Saiu para entrega	Faturado, Acertado, Problema, Cancelado, Agendado	So e aceito se o pedido estiver em carga Em rota.
Agendado	Venda, Faturado, Saiu para entrega, Acertado, Problema, Cancelado	Desvio temporario; tela decide destino conforme NF/carga.
Acertado	Nenhuma; reabertura controlada	Final com entrega concluida.
Problema	Nenhuma; reabertura controlada	Final e exige observacao.
Cancelado	Nenhuma; reabertura controlada	Final.

Criacao

- Exige numero, data de venda, peso maior que zero, cidade, cliente, forma de pagamento e rota resolvida.
- Peso/valor nao podem ser negativos; link de localizacao precisa iniciar com `http://` ou `https://`.
- Prazo e calculado por `add_business_days` usando `sla_limit_days` (padrao 15).
- A cidade/rota e normalizada para maiusculas; o primeiro item sintetico representa o volume/peso total.
- Se o ERP identificar NF, o pedido pode nascer Faturado.

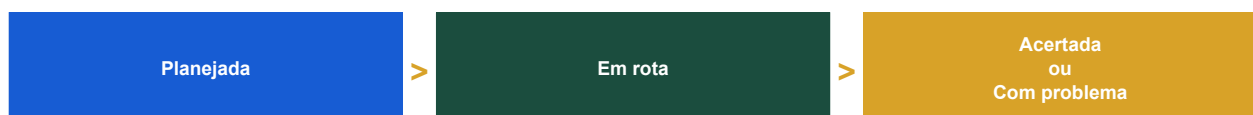
Faturamento e carga

Faturar exige NF e data, bloqueia NF duplicada e usa UPDATE atômico para evitar refaturamento concorrente. A carga exige nome, rota, data, motorista ativo, veiculo ativo, capacidade positiva e ao menos um pedido Faturado ou Saiu para entrega. Rotas nao mistas so aceitam pedidos da mesma rota-base. O peso projetado nao pode superar a capacidade.

Saida e acerto

Ao despachar uma carga Planejada, a carga vira Em rota, os `route_orders` viram Em rota e os pedidos viram Saiu para entrega. O acerto exige checklist de todos os pedidos, data nao anterior a venda e resultado entregue ou problema. Entregues viram Acertado/Entregue; problemas exigem observacao e viram Problema/Com problema. A carga termina Acertada se nao houver problema, ou Com problema se houver.

4.2 Carga/rota



- Sequencia pode ser alterada enquanto a carga nao estiver finalizada/cancelada.
- Adicionar pedido recalcula peso e remove o pedido de outra carga ativa.
- Remover o ultimo pedido de uma carga Em rota e bloqueado.
- `post_route_finish` nao conclui; redireciona conceitualmente para Acerto de carga.
- Cancelar carga devolve pedidos nao finais para Faturado e marca vinculos Cancelado.
- Reabertura de estados finais exige permissao e motivo formal.

4.3 SLA

O prazo usa dias uteis, pulando sabados, domingos e feriados cadastrados. Pedidos finais nao aparecem como risco. Pedidos Agendados recebem tratamento visual neutro. O limiar de risco em codigo e 5 dias; o prazo padrao vem de settings.sla_limit_days. O recalculo em lote incrementa version e grava auditoria.

4.4 Modulos/telas

URL	Modulo	Funcao principal	Permissao GET
/dashboard	Dashboard	Indicadores, pendencias, qualidade e atalhos.	view_dashboard
/orders	Pedidos	Lista, filtros, historico e proximas acoes.	view_orders
/orders/new	Novo pedido	Cadastro manual ou preenchimento ERP.	create_orders
/clients	Clientes	Cadastro mestre e verificacao de duplicidade.	view_clients
/faturamento	Faturamento	Fila Venda/Agendado e baixa de NF.	invoice_orders
/routes	Cargas/rotas	Planejamento e acompanhamento.	view_routes
/routes/new	Nova carga	Selecao de pedidos e capacidade.	create_routes
/load-settlement	Acerto	Checklist e resultado por pedido.	settle_routes
/sla	SLA	Prazos, feriados e recalculo.	view_sla
/drivers /vehicles /route-cities	Catalogos	Motoristas, frota e roteamento-base.	view_* correspondente
/relatorios	Relatorios	KPIs, filtros e CSV.	view_reports
/backup	Backup	Criacao/restauracao SQLite.	view_backup
/settings	Configuracoes	Usuarios, RBAC, tema e parametros.	view_settings
/admin/erp	ERP	Config, teste e sincronizacao.	GOD

5. Banco de dados

O banco ativo e SQLite com WAL. O modelo real contem tabelas criadas por ORM/Alembic e tabelas/colunas adicionadas dinamicamente por `init_db()`, o que produz drift. Para depurar, sempre compare `PRAGMA table_info` do banco alvo com `app.py` e `sqlalchemy_models.py`.

5.1 Entidades operacionais

Tabela	Papel	Campos principais	Qtd.
orders	Pedido central	id, order_number, client_id, seller_id/name, status, datas, pagamento, valor, peso, endereco, rota/cidade, NF, motorista/veiculo, entrega, receipt_photo_at, version	131
order_items	Itens/volume	order_id, produto, quantidade, unidade, peso, notas	131
order_history	Linha do tempo	order_id, user_id, old/new status, action, notes, created_at	574
clients	Clientes	codigo, nome, documento, contatos, cidade, fazenda, endereco, rota, active, version	120
drivers	Motoristas	nome, telefone, documento, veiculo padrao, active, version; pin pode ser adicionado em runtime	8
vehicles	Frota	nome, placa, tipo, capacity/capacity_kg, active, version	6
routes	Carga/viagem	nome, data, driver_id, vehicle_id, status, rota, peso, capacidade, notas, version	29
route_orders	N:N carga-pedido	route_id, order_id, delivery_order, status	112
route_cities	Rota-base	route_name, city, uf, delivery_order, active, version	27
delivery_problems	Ocorrencias	order_id, problem_type, description, created_at	0
delivery_receipts	Canhoto/assinatura	order_id, route_id, image_data, mime_type, digital_signature, recebedor, documento, notas, created_at	0
attachments	Anexos	order_id, file_path, file_type, description, created_at	0

5.2 Seguranca, configuracao e integracao

Tabela	Papel	Campos / observacao
users	Usuarios web	username unico, password_hash, role, active, must_change_password, last_login_at
role_permissions	RBAC por perfil	PK composta role_name + perm, allowed
user_permissions	Override individual	PK composta user_id + perm, allowed
audit_logs	Auditoria global	user_id/name, source_ip, action, module, old/new, notes, created_at
settings	Configuracao persistente	Tema, SLA, capacidade, manutencao e erp_*; precede .env para ERP
holidays	Feriados SLA	date unico, name, created_at
erp_cache_pedidos	Cache ERP	numeropedido, raw_json, erp_synced_at
erp_cache_clientes	Cache ERP	codigocliente, raw_json, erp_synced_at
erp_cache_vendedores	Cache ERP	codigovendedor, raw_json, erp_synced_at
erp_cache_faturamento	Cache ERP	numeropedido, raw_json, erp_synced_at
erp_cache_meta	Estado de sync	key, value, updated_at
alembic_version	DDL	Revision atual aplicada

5.3 Relacionamentos essenciais



orders referencia client, seller, driver e vehicle. order_items, order_history, delivery_problems, delivery_receipts e attachments dependem do pedido. route_orders liga pedido e carga com sequencia e status de campo. users se liga a historico, auditoria e overrides. Varios FKs historicos usam NO ACTION, enquanto filhos de pedido usam CASCADE em partes do esquema.

5.4 Drift confirmado

- delivery_receipts existe no banco e em init_db(), mas não possui classe em sqlalchemy_models.py nem migration dedicada.
- drivers.pin é criado por ALTER TABLE dentro do endpoint change_password; não existe no ORM nem nas migrations e ainda não existia no banco inspecionado.
- delivery_problems possui description, mas a API do motorista tenta usar route_id e notes.
- As tabelas erp_cache_* são criadas dinamicamente pelo conector e não fazem parte do Base ORM.
- O runtime legado rejeita PostgreSQL em create_runtime_connection; a portabilidade exige migração gradual das rotas para SQLAlchemy, não apenas trocar DATABASE_URL.

Regra de depuração

Não conclua que uma coluna existe porque aparece em um teste temporário ou em uma função ALTER TABLE. Reproduza com o mesmo caminho de banco e execute PRAGMA table_info(<tabela>). Para mudanças permanentes, alinhe app.py, SQLAlchemy, Alembic, testes e banco existente.

6. Seguranca, autenticacao e permissoes

A interface web possui sessao, CSRF, origem, rate limit de login e RBAC no backend. A API do motorista, entretanto, e roteada antes dessas barreiras e hoje opera como uma API publica na rede.

6.1 Web administrativo

- Senha PBKDF2-HMAC-SHA256 com 260.000 iteracoes, salt aleatorio e comparacao constante; hashes legados sofrem rehash no login.
- Sessao in-memory com sid aleatorio, expiração padrao de 8 horas e limpeza periodica.
- Cookies sid HttpOnly e SameSite=Lax; Secure depende de LOGISTICA_SECURE_COOKIE.
- Token CSRF por sessao e validacao em POST; checagem de Origin/Referer.
- Rate limit por IP+usuario: janela 600s, 6 falhas, lock 300s por padrao.
- GOD sempre permite; depois override por usuario, role_permissions e fallback do perfil.
- Headers: X-Frame-Options DENY, nosniff, Referrer-Policy, COOP/CORP e CSP.
- Auditoria grava usuario, IP, acao, modulo, entidade, antes/depois e notas.

6.2 Perfis e 32 permissoes

Perfis canonicos: GOD, Admin, Gestor, Faturamento, Expedicao, Motorista, Operador e Consulta. A matriz historica em docs/matriz_permissoes_final.md esta desatualizada (registrava 33 permissoes); o codigo atual define 32. O banco ainda pode ter overrides individuais, por isso a decisao efetiva deve ser verificada em role_permissions e user_permissions.

Grupo	Permissoes
Pedidos	view_orders, create_orders, edit_orders, cancel_orders, invoice_orders, view_financial, register_delivery_problem
Cadastros	view/manage clients, drivers, vehicles e route_catalog
Rotas	view_routes, create_routes, edit_routes, cancel_routes, settle_routes
SLA/relatorios	view_sla, manage_sla, view_reports, export_reports
Administracao	view/manage settings, manage_users, manage_permissions
Backup	view_backup, create_backup, restore_backup
Geral	view_dashboard

6.3 Excecao: API motorista

Fronteira de seguranca ausente

Em do_GET/do_POST, /api/v1/driver/* e tratado antes de require(), same_origin_ok() e validate_csrf(). O cliente nao recebe token. Assim, listar motoristas, cadastrar, trocar PIN, iniciar carga e baixar entrega nao exigem autenticacao de servidor. Isso e relevante tanto para bugs quanto para risco operacional.

7. Integracao ERP

O ERP e fonte read-only. A aplicacao persiste configuracao `erp_*` em settings, com prioridade sobre o `.env`, sincroniza caches locais e pode promover pedidos Venda para Faturado quando detecta NF.

7.1 Drivers e prioridade

Drivers suportados: Oracle/oracledb (padrao interno), SQL Server/pyodbc, MySQL/pymysql e PostgreSQL/psycopg2. A prioridade de configuracao e: settings do banco -> variaveis de ambiente -> default. O objeto ErpConfig e cacheado por 30 segundos.

7.2 Ciclos



- Thread verifica a cada 10s se esta dentro da janela configurada.
- Pendentes Venda sao verificados a cada 60s durante a janela.
- Sync completo respeita `ERP_SYNC_INTERVAL_MIN` (padrao 30min).
- Janela padrao: 07:00-19:00, segunda a sabado.
- `lookup_order` consulta cache e pode fazer live lookup quando necessario/forcado.
- `erp_field_mapper` normaliza datas, numeros, pagamento, endereco, cliente, vendedor e itens.

7.3 Pontos de diagnostico

Sintoma	Verificar
ERP aparece habilitado apesar do <code>.env</code> false	<code>settings.erp_enabled</code> tem prioridade; recarregar config e isolar banco nos testes.
Driver nao instalado	<code>oracledb/pymysql/psycopg2</code> estao em requirements; <code>pyodbc</code> e opcional e requer ODBC no SO.
View/coluna nao encontrada	Admin ERP, <code>qualified()</code> , <code>schema</code> , view configurada e <code>inspect_view()</code> .
Pedido nao atualiza NF	<code>erp_cache_faturamento</code> , <code>lookup_invoice_status</code> , mapeamento e indice unico de <code>invoice_number</code> .
Teste muda conforme maquina	<code>LOGISTICA_IGNORE_DOTENV=1</code> , banco temporario inicializado e <code>ERP_ENABLED=false</code> .

8. Aplicativo do motorista: arquitetura completa

O chamado app Android possui tres camadas: Kotlin nativo, uma PWA servida pelo backend e a API Python. O APK nao reimplementa as telas; ele carrega static/driver_app/index.html em WebView.

8.1 Componentes

Camada	Arquivo-fonte	Responsabilidade
Android nativo	android_app_project/app/src/main/.../MainActivity.kt	WebView, URL do servidor, QR nativo, permissao, camera/galeria, voltar.
Manifest	android_app_project/app/src/main/AndroidManifest.xml	Permissoes, Activity ZXing, FileProvider, cleartext e metadados.
Build	android_app_project/app/build.gradle	SDK, applicationId, versao e dependencias AndroidX/ZXing.
PWA UI	static/driver_app/index.html	Telas, modais, canvas de assinatura e carregamento de scripts.
PWA logica	static/driver_app/driver.js	Estado, login local, chamadas API, foto, assinatura e fila offline.
PWA offline	manifest.json + sw.js	Instalacao e cache basico de assets.
API	app_core/domains/driver_api_dispatch.py	Motoristas, cargas, itens, saida, baixa, canhoto e auto-acerto.
Dados	SQLite	drivers, routes, route_orders, orders, delivery_receipts e delivery_problems.

8.2 Inicializacao nativa

- MainActivity cria WebView programaticamente, sem layout XML.
- Ativa JavaScript, DOM storage, acesso a arquivo/conteudo e playback sem gesto.
- Solicita permissoes e instala WebChromeClient para permissao web e file chooser.
- Le LogisticaPrefs.server_url; se vazio/localhost, abre dialog obrigatorio.
- O usuario escaneia QR, informa HTTPS ou escolhe IP local fixo 192.168.0.100:3000.
- Se a URL nao termina em .html, anexa /static/driver_app/index.html.
- WebView carrega a PWA; erros do frame principal reabrem o dialog de servidor.

8.3 Fluxo PWA



No carregamento, a PWA busca all_drivers, liga eventos online/offline e restaura driver_user do localStorage. O login atual apenas grava {name,pin} localmente; nao chama /driver/login. A lista de cargas filtra por nome. Ao abrir uma carga, carrega pedidos, itens, contatos, peso pendente e comprovante. A baixa envia foto JPEG comprimida, assinatura PNG e dados do recebedor. Se offline ou fetch falhar, guarda o payload no localStorage e tenta novamente no evento online.

8.4 Captura de comprovante

- HTML usa input type=file accept=image/*; o WebChromeClient oferece camera e galeria.
- Android cria arquivo temporario em getExternalFilesDir(Pictures) e entrega content:// ao app de camera via FileProvider.
- Ao retornar, MainActivity envia ao WebView a selecao de galeria ou cameraPhotoPath.
- driver.js le o arquivo, reduz o maior lado para 1280px e converte para JPEG qualidade 0,75.
- Assinatura e capturada em canvas e serializada como data:image/png;base64.
- Backend remove o prefixo data:, decodifica base64 e grava bytes em delivery_receipts.

8.5 Offline

O Service Worker usa network-first e fallback para cache apenas em GET. Entregas offline não são tratadas pelo Service Worker: são payloads JSON completos guardados em `localStorage.offline_deliveries`. O evento online percorre a fila, envia cada item e conserva apenas os que falham. Não existe idempotency key, limite explícito, criptografia, backoff ou painel detalhado de conflito.

8.6 Build e instalação

```
# Android Studio
File > Open > android_app_project
Build > Build Bundle(s) / APK(s) > Build APK(s)

# Saída esperada para debug
android_app_project/app/build/outputs/apk/debug/app-debug.apk

# PWA sem APK
https://SERVIDOR/static/driver_app/index.html
Chrome > Instalar aplicativo / Adicionar a tela inicial
```

O projeto declara `compileSdk/targetSdk 33`, `minSdk 21`, `applicationId br.com.casadocampo.logistica`, `versionCode 1` e `versionName 1.0.0`. Dependências: `core-ktx 1.10.1`, `appcompat 1.6.1`, `material 1.9.0` e `zxing-android-embedded 4.3.0`.

Arquivos de build ausentes no snapshot

`gradlew`, `gradlew.bat`, `gradle-wrapper.jar` e `app/proguard-rules.pro` não existem. O Android Studio pode sincronizar usando seu Gradle embutido e o build debug pode funcionar, mas um build CLI reproduzível e o release precisam completar/validar esses artefatos.

9. Contrato da API do motorista

Todos os endpoints usam `/api/v1/driver`. Os POST recebem JSON. No snapshot, não há token, sessão de motorista ou CSRF; as respostas usam `{ok, message, ...}`.

Endpoint	Entrada	Saída	Comportamento atual
POST /login	driver_name, pin	driver_id, driver_name, vehicle_default	Busca exata e depois LIKE; o PIN é lido mas não validado.
GET /all_drivers	-	drivers[]	Retorna id, nome, telefone, documento e veículo padrão de ativos.
POST /register	name, phone, document, vehicle_default	driver_id, already_existed	Cria motorista ou retorna existente por nome.
POST /change_password	driver_name, new_pin	message	Cria coluna pin em runtime e grava texto puro; login não usa.
GET /routes	query driver_name	routes[] + contagens	Ativas: Em rota, Planejada, Aguardando, Criada; se filtro não acha, retorna todas.
POST /start_route	route_id	status=Em rota	Atualiza carga, vínculos e pedidos sem validar motorista solicitante.
GET /route/{id}	id no path	route + orders + items	Inclui contatos, endereços, valores, peso e flag de canhoto.
POST /deliver	order_id, route_id, recebedor, documento, notas, foto, assinatura, is_problem, problem_type	status, image_saved_in_db, route_auto_settled	Entrega salva canhoto e pode auto-acertar; problema falha no esquema atual.

9.1 Payload de entrega

```
{
  "order_id": 123,
  "route_id": 45,
  "delivered_to": "RECEBEDOR",
  "delivered_document": "DOCUMENTO_SANITIZADO",
  "payment_method": "",
  "final_notes": "observacao",
  "receipt_photo": "data:image/jpeg;base64,...",
  "digital_signature": "data:image/png;base64,...",
  "is_problem": false,
  "problem_type": "Outro motivo"
}
```

9.2 Efeitos de uma entrega normal

- orders.status = Acertado e version incrementa.
- route_orders.status = Entregue para todos os vínculos daquele order_id.
- delivery_receipts anterior é apagado e substituído se houver foto/assinatura.
- orders.receipt_photo_at é atualizado.
- Se todos os pedidos da rota estiverem entregues e nenhum tiver problema, routes.status = Acertada.

10. Diagnosticos confirmados e riscos priorizados

Esta lista foi derivada do codigo atual e, quando indicado, reproduzida em ambiente isolado. Ela serve como backlog tecnico para o ChatGPT, nao como afirmacao de que todos os sintomas ja ocorreram em producao.

Pri.	Achado	Evidencia	Efeito	Direcao
P0	Problema de entrega retorna 500	driver_api_dispatch.py:420 usa route_id e notes; delivery_problems tem description e nao possui route_id.	Reproduzido em banco isolado: 'table delivery_problems has no column named route_id'.	Alinhar SQL/schema e criar teste de is_problem.
P0	PIN nao autentica	driver.js:363-375 nao chama API; /login le pin mas consulta apenas nome; change_password grava pin em texto puro.	Qualquer selecao de motorista entra, mesmo com PIN incorreto.	Projetar autenticao/token, hash do PIN e expirar sessao.
P0	API motorista sem controle de acesso	do_GET/do_POST chama driver API antes de require, same_origin e CSRF.	Operacoes de escrita podem ser chamadas por qualquer cliente com acesso a URL.	Adicionar autenticao/autorizacao e vincular motorista/carga.
P1	QR web tende a ser bloqueado	index.html carrega html5-qrcode de unpkg; CSP permite script-src apenas self; Permissions-Policy define camera=().	Biblioteca externa e getUserMedia podem nao funcionar.	Hospedar lib localmente e definir politica especifica/segura para a PWA.
P1	Duas copias Android divergentes	src/main e app/src/main possuem MainActivity/Manifest diferentes.	Edicao no caminho errado nao entra no APK.	Remover/arquivar copia antiga e documentar fonte de verdade.
P1	URI da foto de camera pode falhar	Camera recebe content://, mas callback ao WebView usa file: + caminho absoluto.	WebView pode nao conseguir ler ou pode retornar selecao vazia.	Preservar content URI e usar parseResult/ClipData corretamente.
P1	URL customizada cross-origin e inconsistente	PWA aceita custom_server_url; CSP connect-src self e servidor nao implementa CORS.	Fetch para outro host/origem e bloqueado; no APK a URL nativa e diferente do localStorage web.	Usar mesma origem ou implementar contrato CORS/allowlist consciente.
P1	Fallback expoe todas as cargas	GET /routes retorna todas as cargas ativas se o filtro do motorista nao encontra nada.	Motorista pode operar carga alheia por erro de nome/atribuicao.	Retornar vazio ou exigir associacao por id/token.
P1	Navegacao WebView sem allowlist	shouldOverrideUrlLoading carrega toda URL dentro da WebView; QR aceita URL arbitraria.	Links tel/WhatsApp/maps podem falhar; URL maliciosa pode abrir no contexto do app.	Tratar esquemas externos com Intent e restringir hosts internos.
P2	Fila offline sem idempotencia/limite	Payload base64 e guardado em localStorage; retry nao possui chave unica.	Quota pode estourar; reenvio pode sobrescrever comprovante ou duplicar ocorrencia.	IndexedDB, limites, hash/idempotency key, estado e backoff.
P2	Atualizacoes amplas por order_id	deliver atualiza route_orders WHERE order_id=? sem restringir route_id.	Historico duplicado/inconsistente pode afetar cargas antigas.	Validar vinculo route_id/order_id e atualizar o registro correto.
P2	Drift ORM/Alembic/runtime	delivery_receipts, pin e caches nao estao integralmente versionados.	Banco novo, teste e producao podem ter schemas diferentes.	Criar migrations unicas e remover ALTER TABLE de endpoint.
P2	Permissoes Android excessivas	RECORD_AUDIO, storage legado, grant de todos WebResource; universal file URL e cleartext habilitados.	Superficie maior e prompts desnecessarios.	Minimizar permissoes e filtrar PermissionRequest por origem/recurso.
P2	Build Android nao totalmente reproduzivel	Wrapper scripts/jar e proguard-rules ausentes.	CLI/release pode depender da maquina/Android Studio.	Gerar wrapper, criar regras, assinatura e pipeline de build.
P3	Service Worker com versionamento manual	CACHE_NAME driver-app-v1 e lista fixa; HTML depende de CDN externa.	Atualizacoes podem ficar inconsistentes e QR nao funciona offline.	Versionar assets por release e eliminar dependencia externa.

10.1 Exemplo do erro P0 reproduzido

```
POST /api/v1/driver/deliver (is_problem=true)
HTTP 500
Erro interno ao salvar entrega:
table delivery_problems has no column named route_id

Schema real:
delivery_problems(order_id, problem_type, description, created_at)

SQL atual da API:
INSERT INTO delivery_problems(order_id, route_id, problem_type, notes, created_at)
```

10.2 Ordem recomendada de correcao



11. Testes e qualidade

A suite unittest contem 51 testes. Em ambiente isolado com schema inicializado, todos passaram. A primeira execucao herdou .env/settings do ERP e falhou, confirmando que a reproducao precisa isolar configuracao e banco.

11.1 Resultado verificado

Execucao	Resultado	Interpretacao
Ambiente local herdando .env/banco	43 passaram, 8 falharam	Testes de defaults ERP foram contaminados por settings do banco; driver Oracle ausente tambem apareceu.
Banco temporario vazio sem init_db	49 passaram, 2 erros	Dois testes esperavam tabelas ja existentes.
Banco temporario inicializado + dotenv ignorado + ERP false	51/51 passaram	Comando reproduzivel e isolado.
Repro manual is_problem	HTTP 500 confirmado	Caminho nao coberto pela suite do motorista.

11.2 Comando isolado

```
# PowerShell - use um banco temporario dedicado
$env:LOGISTICA_IGNORE_DOTENV='1'
$env:DATABASE_URL='sqlite:///tmp/test_isolated.sqlite3'
$env:FLASK_ENV='testing'
$env:SECRET_KEY='isolated-test-secret'
$env:ERP_ENABLED='false'
python -c "import app; app.init_db()"
python -m unittest discover -s tests -v
```

11.3 Cobertura relevante

- API motorista: cadastro/lista, troca de PIN, saida, itens, entrega com foto e auto-acerto.
- Nao ha teste de deliver com is_problem=true, autenticacao real do PIN, CORS/CSP, WebView ou fila offline.
- ERP: config, defaults, lookup desabilitado, mapeamento, UI admin e rotas HTTP.
- Core: banco/runtime, backup, data em lote, agendamento e gestao de usuarios.
- Ferramental adicional inclui chaos audit, stress smoke, Playwright E2E, auditoria de layout e gate de release.

11.4 Testes que devem acompanhar os bugs do app

Teste	Assercao minima
driver_problem_delivery	POST is_problem grava description, atualiza somente o vinculo correto e retorna 200.
driver_login_pin	PIN incorreto 401; correto gera token; PIN armazenado com hash.
driver_authorization	Token de outro motorista nao inicia/baixa carga alheia.
delivery_idempotency	Mesmo idempotency_key nao duplica problema nem recibo.
schema_migration	Banco 0003 antigo sobe para novo head sem ALTER em endpoint.
pwa_headers	CSP permite apenas assets locais esperados e camera no path do app.
android_instrumented	Camera, galeria, QR, tel/WhatsApp e process death preservam callback/estado.

12. Configuracao, instalacao e operacao

A configuracao e lida de .env, com APP_* tendo precedencia sobre aliases LOGISTICA_* para host/porta. SECRET_KEY e obrigatoria em producao, salvo override explicitamente inseguro.

12.1 Variaveis essenciais

Variavel	Default	Funcao
FLASK_ENV	production	Controla modo e obrigatoriedade de secret.
DEBUG	false	Em producao exige LOGISTICA_ALLOW_PROD_DEBUG=1.
SECRET_KEY	obrigatoria	Nunca enviar em logs/prompts.
APP_RUNTIME	flask	flask ou legacy.
APP_HOST / APP_PORT	127.0.0.1 / 3000	Use 0.0.0.0 para LAN.
LOGISTICA_LEGACY_PROXY_PORT	APP_PORT+1000	Porta interna do bridge.
DATABASE_URL	sqlite:///data/...	Runtime legado aceita apenas SQLite na pratica.
LOGISTICA_SECURE_COOKIE	0	Use 1 sob HTTPS confiavel.
LOGISTICA_SESSION_MAX_AGE	28800	Segundos.
LOGISTICA_LOGIN_*	600 / 6 / 300	Janela, max falhas e lock.
LOGISTICA_MAX_WORKERS	40	Limite do servidor legado.
LOGISTICA_REQUEST_TIMEOUT	30	Socket/proxy.
ERP_*	desabilitado por default	Settings erp_* do banco podem sobrepor.

12.2 Instalacao Windows

```
python -m pip install -r requirements.txt
python -m alembic upgrade head
python tools/alembic_bootstrap.py          # banco existente, se necessario
python run.py

# Operacao assistida
iniciar.bat
instalar_servidor.bat                    # como Administrador

# Validacao
http://127.0.0.1:3000/healthz
python tools/db_integrity_audit.py
```

12.3 Backup e restauracao

- Tela Backup cria snapshot SQLite consistente pela API backup() e mantem os 7 mais recentes.
- Restore exige permissao restore_backup, texto RESTAURAR e motivo; cria pre_restore antes de substituir.
- backup_automation.py suporta backup, verify e all; a verificacao restaura em arquivo temporario e executa integrity_check.
- PostgreSQL usa pg_dump/pg_restore por tools/postgres_backup_restore.py, mas o runtime de regras ainda precisa migracao SQLAlchemy.
- RTO documentado para SQLite local: 10-20 minutos; RPO ate o ultimo backup valido.

12.4 Logs e monitoramento

Arquivo/endpoint	Uso
/healthz	status, hora, host, porta e sessoes ativas; publico.
logs/server_errors.log	Erros legiveis.
logs/server_errors.jsonl	Erros estruturados e stack/contexto.
logs/runtime.log / runtime.err.log	Processo do servidor/monitor.
logs/watchdog.log / uptime_monitor.log	Supervisao e disponibilidade.
logs/automation_status.json	Ultimo backup/verify e automacoes.
logs/audit_reviews/	Revisao semanal de acoes criticas.

13. Guia de diagnostico do app Android

Colete primeiro a camada que falhou. Um erro visual pode estar no JS; um HTTP 500 esta no Python/SQLite; camera/arquivo pode estar no Kotlin; conexao pode estar em URL, TLS, CSP ou proxy.

13.1 Arvore de decisao

Sintoma	Primeira verificacao
App nao abre servidor	Dialog URL, DNS/IP, HTTPS/certificado, INTERNET, cleartext, healthz e onReceivedError.
Lista de motoristas/cargas vazia	Chrome/WebView console, Network, endpoint all_drivers/routes e custom_server_url.
PIN aceita qualquer valor	Comportamento atual confirmado: login e local e API ignora pin.
Foto nao aparece	onShowFileChooser, FileProvider, retorno content/file URI, previewPhoto e tamanho/quota.
Entrega retorna erro	Response body de /deliver, server_errors, schema SQLite e payload is_problem.
QR nao abre	CSP, Permissions-Policy, carregamento de html5-qrcode e permissao CAMERA.
WhatsApp/mapa nao abre	shouldOverrideUrlLoading e tratamento de tel:/https externo.
Offline nao sincroniza	navigator.onLine, localStorage.offline_deliveries, base URL, resposta ok e quota.
APK nao reflete codigo	Confirmar edicao sob app/src/main e rebuild/instalacao da versao correta.

13.2 Evidencias para coletar

- Modelo do aparelho, versao Android, versao do Android System WebView e versao do APK.
- URL sanitizada (host e path, sem tokens), rede usada e resultado de /healthz.
- Passos exatos, resultado esperado, resultado atual, horario e ID anonimo do pedido/carga.
- Logcat filtrado pelo package br.com.casadocampo.logistica.
- Console e Network via chrome://inspect para a WebView/PWA.
- Status HTTP e JSON de resposta da API; stack correspondente no server_errors.jsonl.
- PRAGMA table_info apenas das tabelas envolvidas; nunca anexar banco inteiro sem sanitizacao.

13.3 Comandos uteis

```
# Android conectado por ADB
adb devices
adb logcat --pid=$(adb shell pidof br.com.casadocampo.logistica)

# Backend
curl http://SERVIDOR:3000/healthz
python tools/db_integrity_audit.py

# Schema focal (executar em copia/teste se houver duvida)
python -c "import sqlite3; c=sqlite3.connect('data/logistica_casa_do_campo.sqlite3'); print(c.execute('pragma table_info(delivery_problems)').fetchall())"
```

Privacidade

Antes de colar Logcat, JSON, HTML, SQL ou logs no ChatGPT, remova nomes, documentos, telefones, enderecos, credenciais, host interno quando sensivel, fotos, assinaturas e raw_json do ERP. Use IDs ficticios mantendo apenas tipos, status e estrutura.

14. Pacote pronto para usar no ChatGPT

Use o texto abaixo como contexto inicial. Depois anexe somente os arquivos do caminho do bug e as evidencias sanitizadas. Para correcao, peça primeiro causa raiz e plano de testes; depois solicite o patch.

14.1 Prompt-base

Estou corrigindo o sistema "Logistica Casa do Campo", snapshot 25/08/2026, versao v2.6.0-20260820, commit 3366ac8.

Arquitetura real:

- run.py inicia Flask/Waitress.
- app_core/app_factory.py encaminha tudo a um servidor legado interno.
- app.py contem as regras, SQL, sessao e a maior parte das telas.
- O app Android e uma WebView; a UI/logica fica em static/driver_app.
- A API fica em app_core/domains/driver_api_dispatch.py.
- Banco ativo: SQLite WAL; schema tambem e alterado por init_db().

Fonte Android compilada:

android_app_project/app/src/main/java/br/com/casadocampo/logistica/MainActivity.kt
(nao usar a copia antiga em android_app_project/src/main).

Antes de propor mudanca:

1. Identifique a camada do bug: Kotlin, PWA JS/HTML, API Python ou SQLite.
2. Trace request -> endpoint -> SQL -> efeito de status.
3. Compare o SQL com o schema/migration real.
4. Preserve o fluxo Venda -> Faturado -> Saiu para entrega -> Acertado/Problema.
5. Nao reduza autenticacao, RBAC, CSRF ou auditoria.
6. Inclua testes de regressao e instrucoes de reproducao isolada.

Bug observado:

[DESCREVA PASSOS, ESPERADO, ATUAL, HORARIO E HTTP/ERRO]

Arquivos anexados:

[LISTE OS ARQUIVOS]

Quero: causa raiz com evidencias por linha, impacto, patch minimo e seguro, migrations necessarias, testes unitarios/integracao e checklist Android.

14.2 Arquivos por categoria

Bug	Anexar
Problema de entrega 500	driver_api_dispatch.py, trecho de init_db em app.py, sqlalchemy_models.py, migrations e schema table_info.
Login/PIN	driver.js, index.html, driver_api_dispatch.py e proposta de migration drivers.
Camera/galeria	MainActivity.kt compilada, Manifest, file_paths.xml, driver.js previewPhoto e Logcat.
QR Code	index.html, driver.js startQrScanner, headers CSP/Permissions-Policy de app.py e console WebView.
Cargas erradas	driver.js loadRoutes, endpoint /routes, route/driver rows anonimizadas e SQL de associacao.
Offline	driver.js submitDelivery/syncOfflineQueue, SW, quota/console e respostas /deliver.
Build APK	Gradle raiz/app, settings, wrapper properties, tree do projeto e erro completo do Gradle.
ERP	erp_connector.py, erp_field_mapper.py, config sanitizada, status do cache e stack sem raw_json.

14.3 Formato de relato

Titulo:
Ambiente (Android/Web/Servidor):
Versao/commit:
Pre-condicao:
Passos numerados:
Resultado esperado:
Resultado atual:
HTTP status + resposta sanitizada:
Logcat/console/server log sanitizado:
ID ficticio de pedido/carga:
Reproduz sempre? (sim/nao, frequencia):
Comecou apos qual alteracao?:

15. Plano de manutencao recomendado

O sistema esta operacional e possui boa cobertura de fluxos centrais, mas o app do motorista cresceu por adicoes dinamicas. O proximo ciclo deve transformar esses caminhos em contratos versionados e testaveis.

15.1 Curto prazo

- Corrigir schema/SQL de problema do motorista com migration e teste.
- Implementar autenticacao real do motorista e autorizacao por carga.
- Remover retorno de todas as cargas como fallback silencioso.
- Ajustar CSP/Permissions-Policy ou hospedar html5-qrcode localmente.
- Consertar o retorno de URI de camera e testar Android 8, 11, 13+.

15.2 Medio prazo

- Versionar delivery_receipts e pin em Alembic/ORM; eliminar ALTER TABLE em request.
- Adicionar token curto/refresh ou sessao de motorista e hash do PIN.
- Migrar fila offline para IndexedDB com idempotencia e UI de conflitos.
- Consolidar copia Android e tornar tools/build_android_apk.py fonte geradora testada ou removê-lo.
- Completar Gradle wrapper, release signing e CI de APK.
- Separar progressivamente app.py em servicos/repositories sem alterar URLs.

15.3 Criterios de pronto para cada bug

Criterio	Evidencia
Causa reproduzida	Teste ou roteiro falha antes e passa depois.
Banco versionado	Alembic upgrade em banco antigo e novo; downgrade conservador quando aplicavel.
Sem regressao	51 testes atuais + novos testes do bug passam isolados.
Seguranca preservada	Autenticacao/autorizacao, PII, CSRF/CORS/CSP revisados.
Android verificado	APK debug limpo, instalacao nova e upgrade testados; Logcat sem execucao.
Offline verificado	Online, sem rede, retomada, reenvio e duplicidade testados.
Operacao atualizada	Documentacao, VERSION, gerador Android e runbook coerentes.

A. Apendice - Inventario de rotas

Resumo de URLs estaveis e acoes importantes. Rotas de detalhe usam IDs no path. Todas as rotas web, exceto login, static, healthz e driver API, exigem sessao.

Metodo	Path	Funcao
GET	/login /logout /force-password	Autenticacao e troca obrigatoria.
GET	/healthz	Healthcheck publico.
GET	/events	SSE autenticado (atualmente envia connected e encerra).
GET	/orders/erp-lookup	Busca e mapeia pedido ERP.
GET/POST	/orders/{id}/edit	Edicao com version/updated_at.
POST	/orders/{id}/invoice deliver problem status reopen delete	Ciclo do pedido.
POST	/clients/drivers vehicles route-cities/{id}/action]	CRUD/toggle de catalogos.
GET/POST	/routes /routes/new /routes/{id}	Lista, cria e detalha carga.
POST	/routes/{id}/dispatch update sequence add remove cancel reopen delete	Operacao de carga.
POST	/load-settlement/{id}/set-date finish	Data em lote e acerto.
GET	/load-settlement/{id}/print-report	Relatorio de carga.
POST	/sla/holiday /sla/recalculate	Feriadados e SLA.
GET	/relatorios/export	CSV.
POST	/backup/create /backup/restore	Backup/restore SQLite.
GET/POST	/admin/erp/status test sync]	Administracao ERP exclusiva GOD.
POST	/settings/...	Usuarios, perfil, permissoes e parametros.
GET/POST	/api/v1/driver/*	API PWA/APK; atualmente sem sessao.

B. Apendice - Comandos de operacao

Execute comandos destrutivos apenas em ambiente de teste ou com backup validado. Nunca restaure por cima do banco em uso.

Objetivo	Comando
Executar	python run.py
Waitress	waitress-serve --host=0.0.0.0 --port=3000 run:app
Legado	set APP_RUNTIME=legacy && python run.py
Migrations	python -m alembic upgrade head
Bootstrap	python tools/alembic_bootstrap.py
Testes	python -m unittest discover -s tests -v
Gate	python tools/regression_release_gate.py
Integridade	python tools/db_integrity_audit.py
Backup	python tools/backup_automation.py --mode backup --keep-max 7
Verificar backup	python tools/backup_automation.py --mode verify
Auditoria semanal	python tools/audit_log_weekly_review.py --days 7
Baseline	python tools/freeze_baseline.py --name rc_interno_YYYYMMDD --notes descricao
Go-live	powershell -ExecutionPolicy Bypass -File tools/run_checklists.ps1 -Mode go-live -FullGate
Monitor	powershell -ExecutionPolicy Bypass -File tools/uptime_monitor.ps1
Android generator	python tools/build_android_apk.py

Conclusao

O sistema combina uma aplicacao web madura com uma migracao arquitetural ainda incompleta e um app de campo em tres camadas. A chave para consertar bugs sem criar regressao e rastrear cada sintoma ate a camada certa e validar o schema real. O problema de entrega do app ja possui causa raiz confirmada; autenticacao de PIN, fronteira da API, CSP/QR, URI de camera e offline devem ser tratados como proximas prioridades.

Resumo para a proxima conversa com o ChatGPT

Envie este PDF como contexto, descreva um bug por vez, anexe apenas os arquivos da camada envolvida e exija: causa por linha, patch minimo, migration quando houver schema, teste que falha antes/passa depois e checklist Android/servidor. Nao envie o .env, o banco, fotos, assinaturas ou raw_json do ERP.

Documento gerado a partir do repositorio local em 25/08/2026. Versao v2.6.0-20260820, commit 3366ac8. Nenhuma alteracao funcional foi realizada no sistema durante a elaboracao.